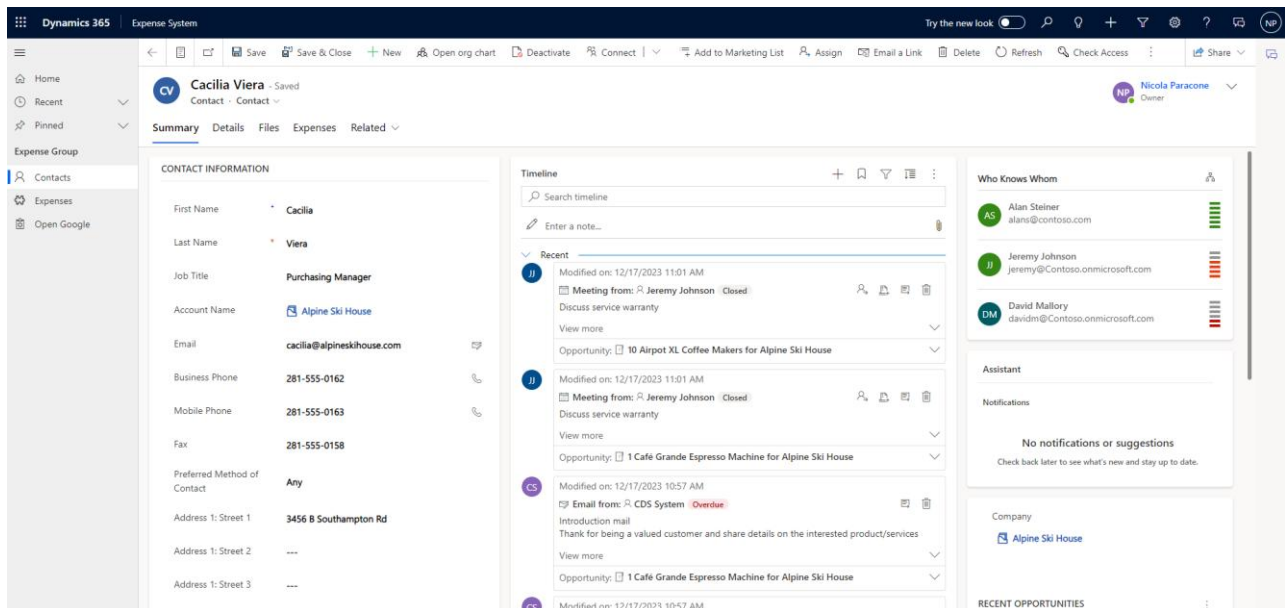


Creation of Business Rules & JavaScript Custom Codes

Microsoft gives us a lot of functionalities, but if we want to have our own functionalities, we can build them from the UI only. If we have the need of extending Microsoft functionalities a little more, we can use JavaScript for client side and C# for server side. Client-side scripting works when the user is active on the screen (what should happen if the user clicks on something, or a page is loaded, or saved?).

Assume that we have a contact open, and some fields change or the contact is reloaded. Should anything be displayed to the user, such as a pop-up advertising, or should something else happen? Such actions can be coded in a JavaScript custom code.



JavaScript in Dynamics 365:

JavaScript is often used for client-side customization and automation within Dynamics 365. It is commonly used in web resources, form scripting, and ribbon customizations. JavaScript can be utilized to enhance the user interface, validate data, and perform actions based on user interactions. It is executed in the context of the user's browser, providing dynamic and responsive user experiences.

C# in Dynamics 365:

C# is primarily used for server-side development in Dynamics 365. Custom plugins, workflow assemblies, and custom actions are typically implemented using C#. These components run on the server and can be triggered by various events, such as record creation, update, or deletion. C# code allows for more complex business logic, integration with external systems, and data manipulation within the server-side context of Dynamics 365.

We talked about background and real-time workflows, and pointed out that the changes due to a background flow are visible only after refresh. With server-side code we have both asynchronous coding, which does not affect user's performance, and synchronous coding, which in principle does affect user's experience. However, it auto refreshes so quickly that the user's experience and performance are not influenced.

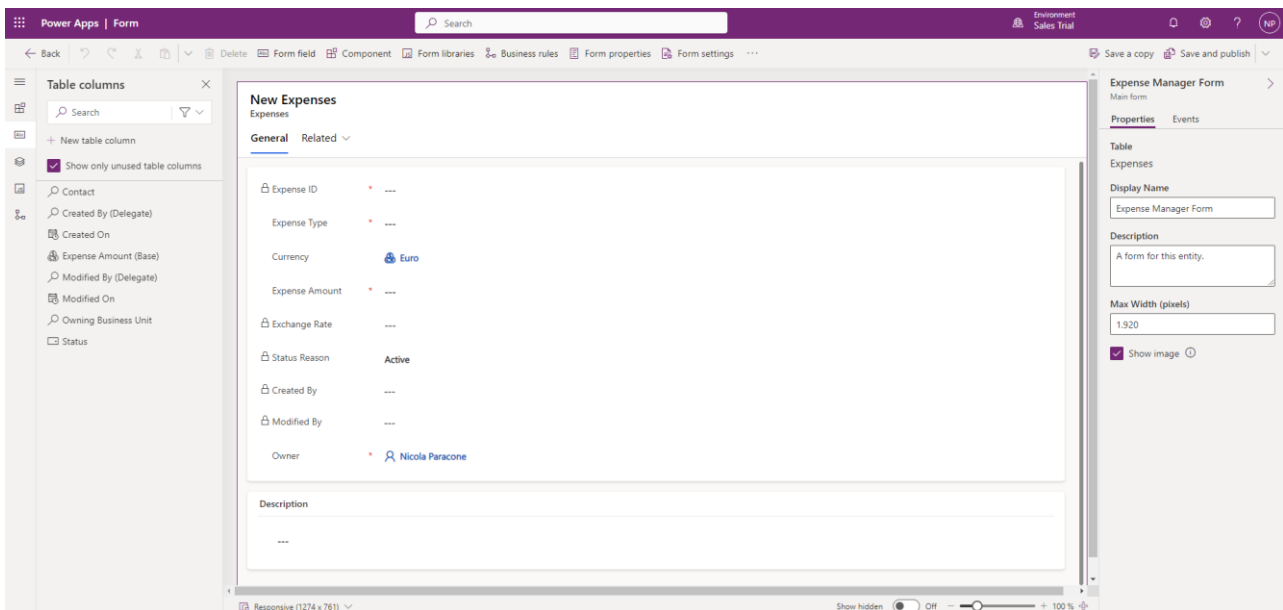
In JavaScript/client side we have multiple trigger formats:

On load

On change of fields

These two are the most important triggers that we will be using.

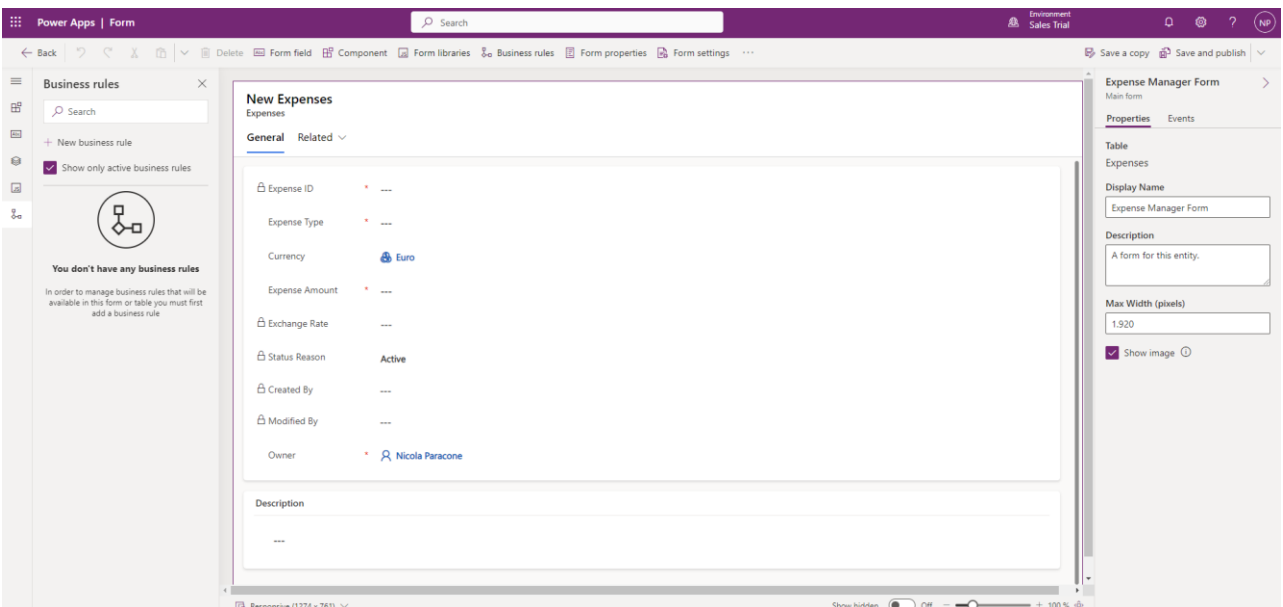
Go to Power Apps, open the Demo solution and go to Table>Contact>Expenses. Open for example the Expense Manager Form.



Here we have Form Properties and Form Libraries. Form Libraries holds the JavaScript web resources that we have added, or will add, to this form. Then we have the Business Rules, which are another tool worth to be discussed immediately.

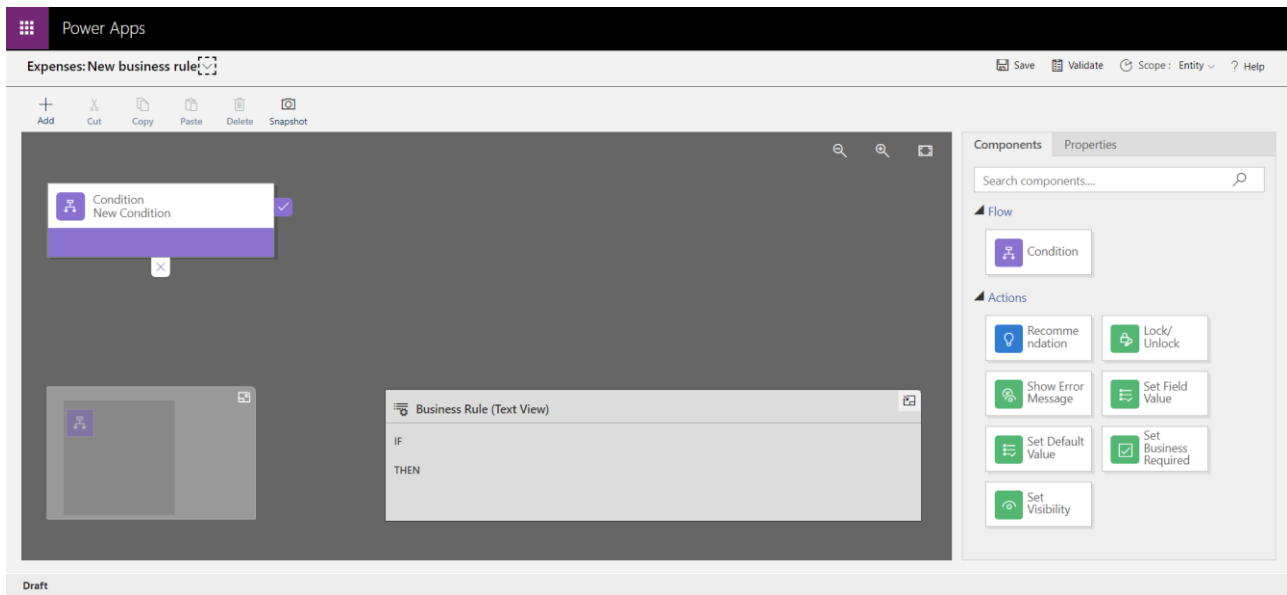
1. Business rules

Business rules are a form of “client-side scripting” with basic control.

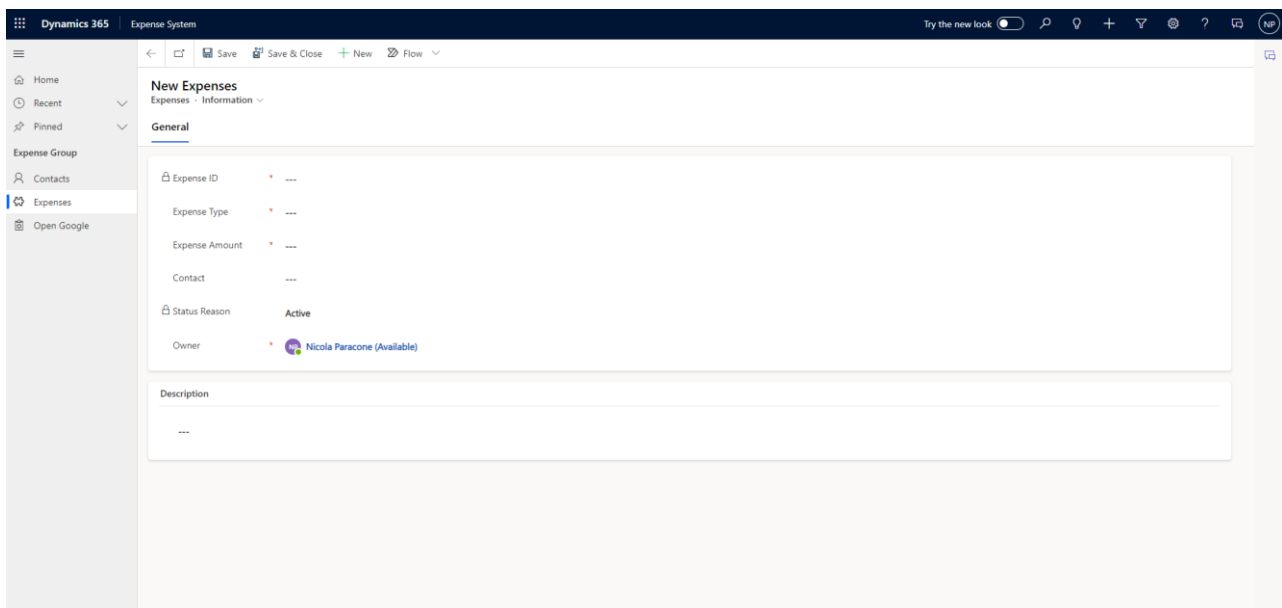


Let us create a new business rule by clicking on + New business rule.

This is the designer which will pop up.



An example of business rule to be implemented could be the following: in a new expense we want the Expense Amount field to be autocompleted based on Expense Type.



This kind of things can be done with JavaScript as well, but most of the times we will be using business rules (we should always try to minimize our coding).

The Scope menu, in the designer, determines the entity to which the business rules will be applied (in our case we select the Expense Manager Form). The action Lock/Unlock serves to make unchangeable (read-only) a field after it has been populated according to the value of another field (or another condition). Using Set Default Value we have the possibility to choose a default value for one or more fields. Set Business Required is used to make a field mandatory. Set Visibility serves to dynamically hide and show one or more fields. All these actions are performed dynamically and, as we already said, they can be implemented also using JavaScript. Using plug-ins, however, we cannot hide and show fields dynamically, make fields mandatory, or show error messages.

Make sure that the Required field associated with Expense Amount is set to Optional. We define a business rule to auto populate Expense Amount with a value based on a precise Expense Type option we have selected (Flight in this example).

If Expense Type = Flight:

The screenshot shows the 'Expenses: New business rule' interface in Power Apps. The main canvas displays a 'Condition' block (New Condition) and a 'Business Rule (Text View)' block. The 'Business Rule (Text View)' block shows the following logic:

```
IF  
  Expense Type equals "Flight"  
THEN
```

The right-hand pane shows the 'Properties' tab for the 'Condition' block. The 'Display Name' is 'New Condition'. The 'Entity' is 'Expenses'. The 'Rules' section shows a single rule with the following properties:

- Source: Entity
- Field: Expense Type
- Operator: Equals
- Type: Value
- Value: ☒ Flight, ☐ Food, ☐ Hotel, ☐ Hospital

The 'Apply' button is visible at the bottom right of the right-hand pane.

then we want to set the value of Expense Amount to 5000 euros.

The screenshot shows the 'Expenses: New business rule' interface in Power Apps. The main canvas displays a 'Condition' block (New Condition) and a 'Set Field Value' block (New Action). The 'Business Rule (Text View)' block shows the following logic:

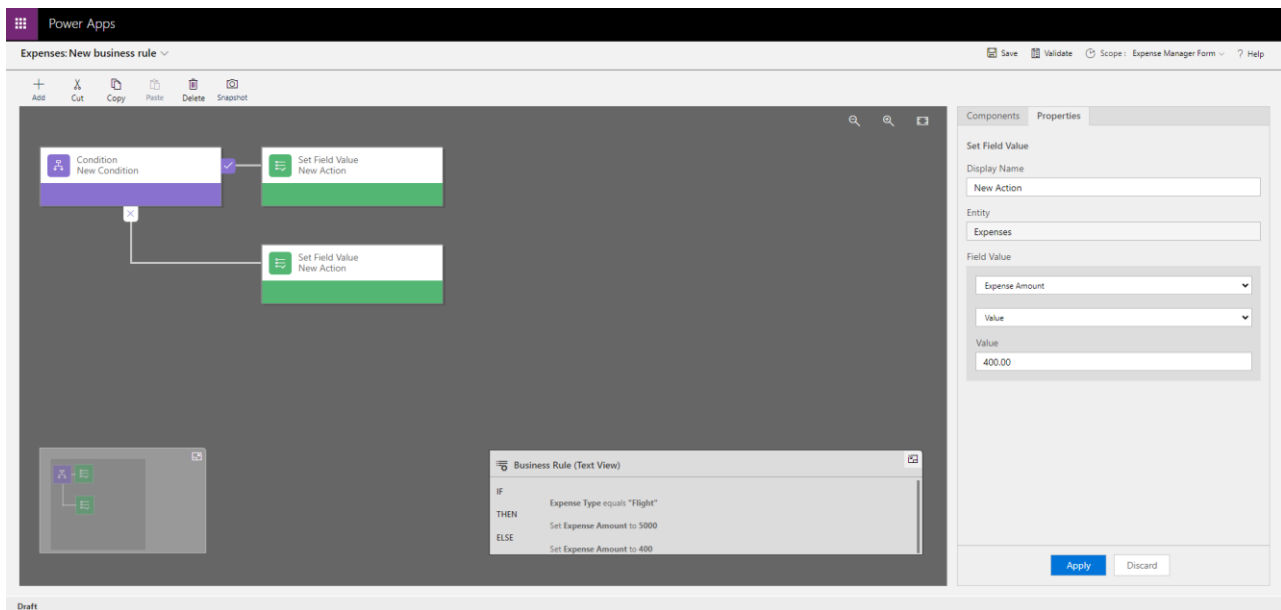
```
IF  
  Expense Type equals "Flight"  
THEN  
  Set Expense Amount to 5000
```

The right-hand pane shows the 'Properties' tab for the 'Set Field Value' block. The 'Display Name' is 'New Action'. The 'Entity' is 'Expenses'. The 'Field Value' section shows the following properties:

- Field Value: Expense Amount
- Value:

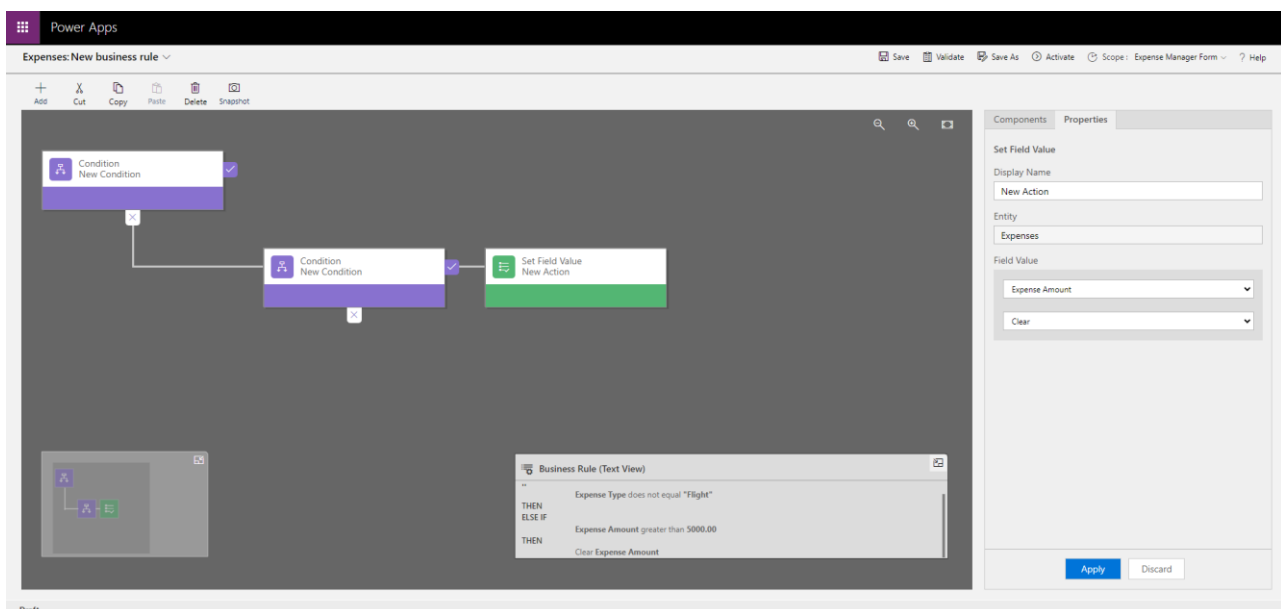
The 'Apply' button is visible at the bottom right of the right-hand pane.

If it is not, we set it equal to 400 euros. Remember to always click on Apply.



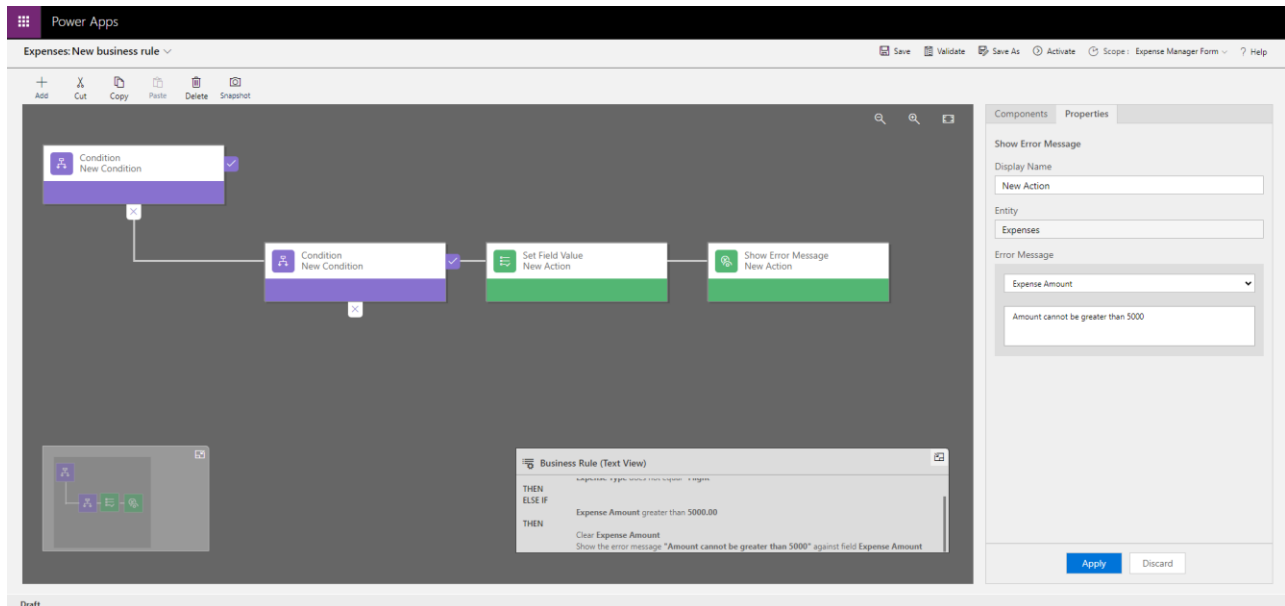
Save and click on Activate. Then come back to Power Apps and click on Save and Publish. Test the new feature in a new expense.

Deactivate the business rule we have just created and delete the two Set Field Value boxes. We keep the same condition as before, but we add another condition. We modify the first one to If Expense Type != Flight and insert another condition saying If Expense Amount > 5000. Hence, we set the field value Expense Amount to Clear.

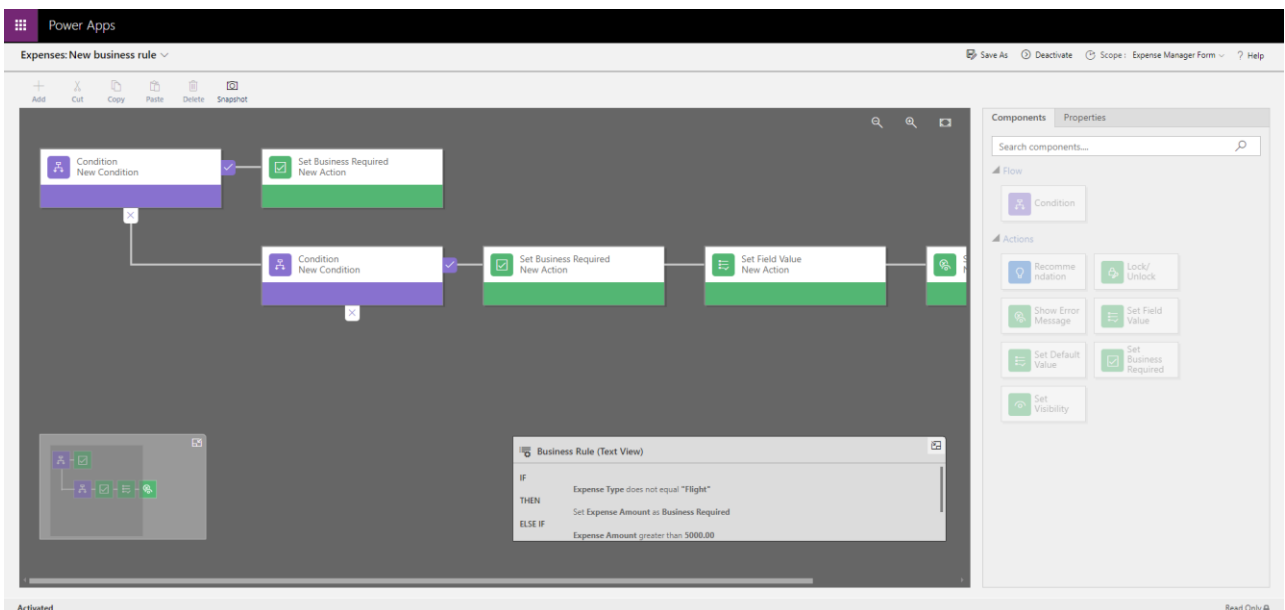


In other words, if Expense Type is equal to Flight and the Expense Amount is larger than 5000, then clear that field value.

Then we want to show an error message saying "Amount cannot be greater than 5000".



This, however, will throw an error. This is because we need to add something for when the first condition is true. For this we put a Set Business Required and make the Expense Amount field mandatory.



Notice that we also must put a Set Business Required, specifying a Not Business Required on Expense Amount, after the second condition.

The following is an example of what we will see in Dynamics 365 if we set a value higher than 5000 for a Flight expense.

The screenshot shows the 'New Expenses' form in Dynamics 365. The form is titled 'New Expenses - Unsaved' and 'Expenses - Expense Manager Form'. The 'General' tab is selected. The form contains the following fields:

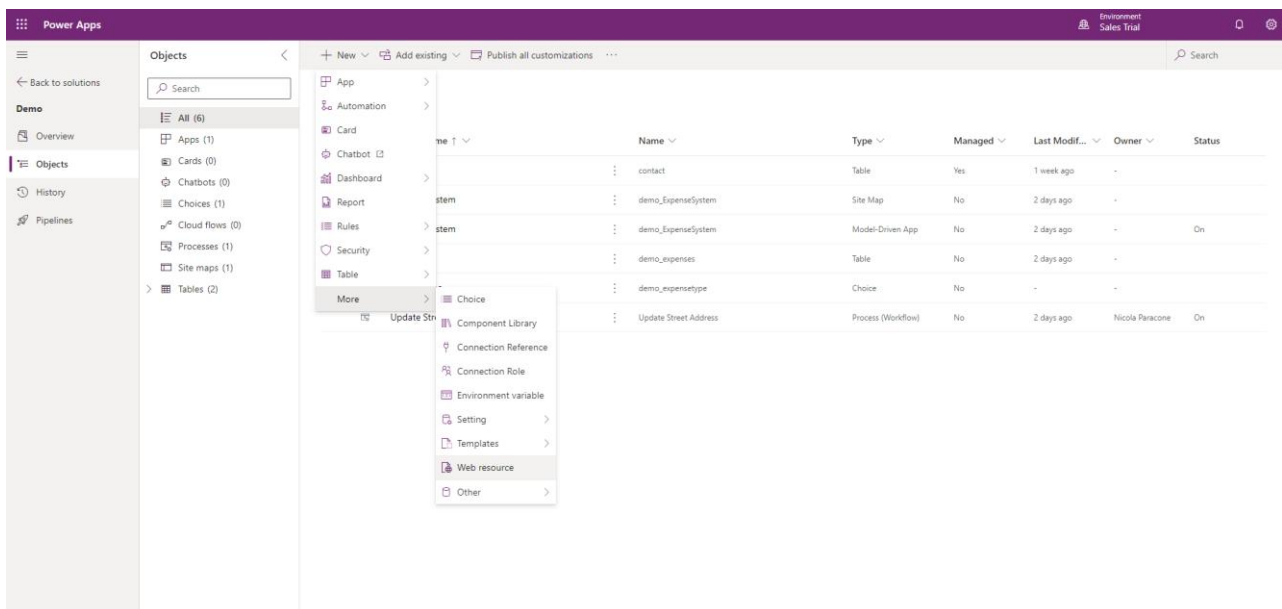
- Expense ID: ---
- Expense Type: Flight
- Currency: Euro
- Expense Amount: ---
- Exchange Rate: ---
- Status Reason: Active
- Created By: ---
- Modified By: ---
- Owner: Nicola Paracone (Available)

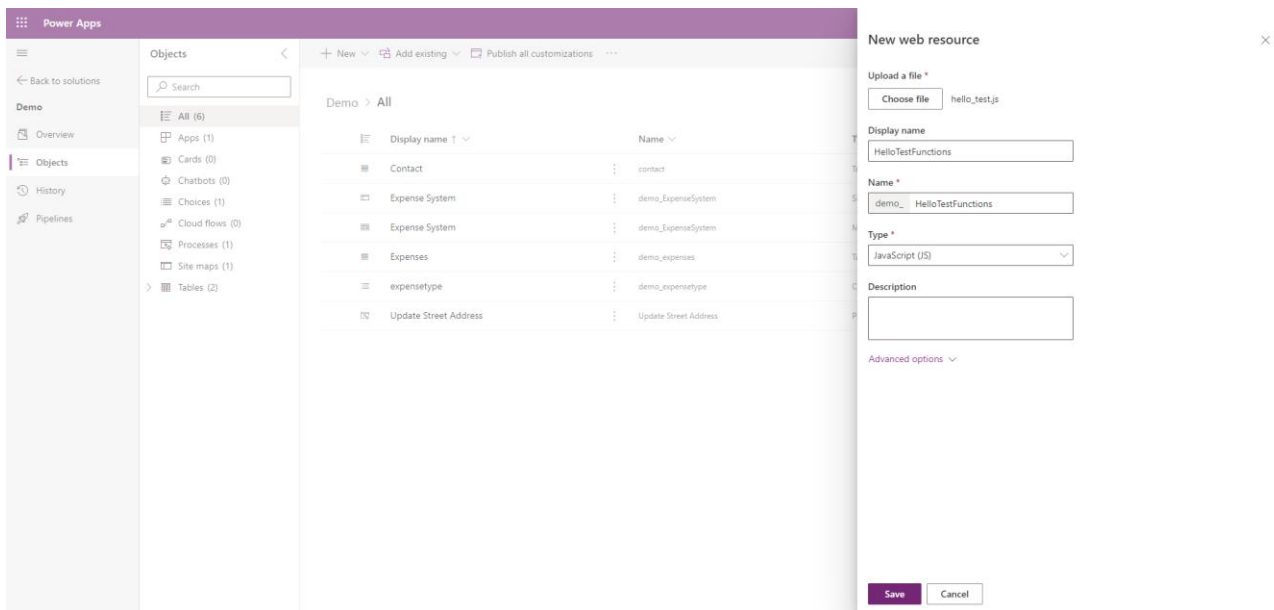
A red error message is displayed below the Expense Amount field: 'Expense Amount: Amount cannot be greater than 5000'.

Now we deactivate the business rule and check what JavaScript offers.

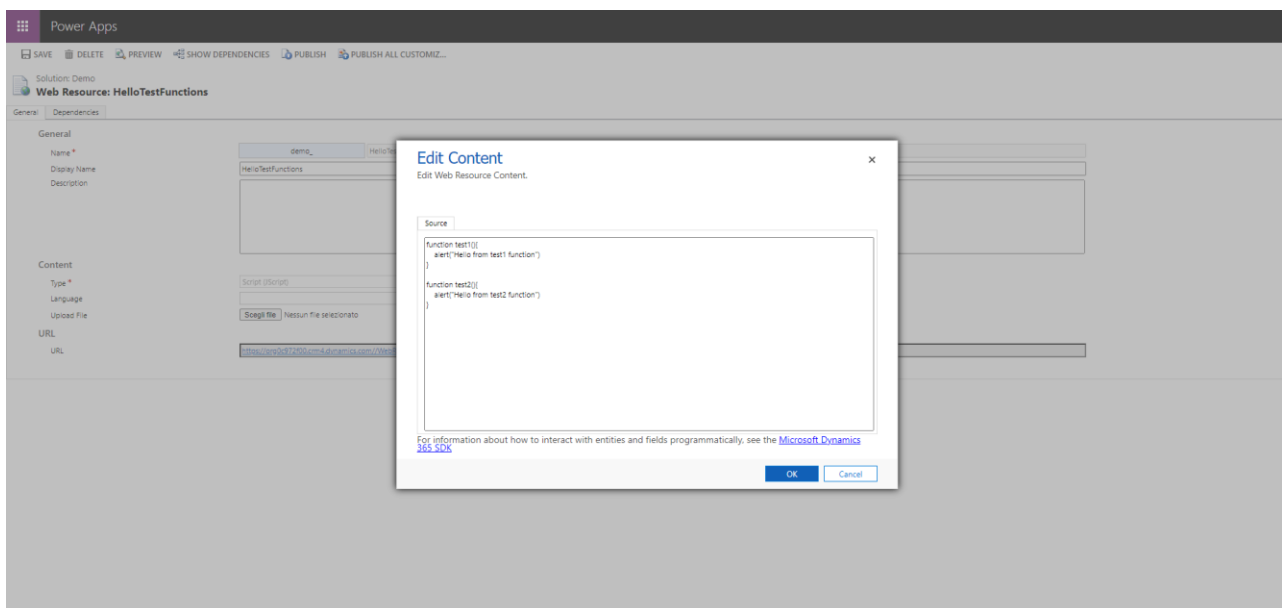
2. Adding JavaScript custom code

We create the same logic obtained with business rules, but this time using a JavaScript code. In Power Apps go to Demo>All, click + New and under More choose Webresource.





Here we must add a file containing the code we want to run. Notice that we are not provided with an editor, and therefore we have to write our code somewhere else and then import it. In the reality, however, once the web resource has been created and saved, we can edit it in classic mode. There we have an editor.



Now go back to the form we were editing, click on Form libraries and then on + Add library.

The screenshot shows the Power Apps Form editor interface. On the left, the 'Form libraries' pane is open, displaying a search bar and a '+ Add library' button. The main area shows a form titled 'New Expenses' with various fields: Expense ID, Expense Type, Currency (set to Euro), Expense Amount, Exchange Rate, Status Reason (set to Active), Created By, Modified By, Owner (set to Nicola Paracone), and a Description field. On the right, the 'Expense Manager Form' pane is visible, showing a '+ Add library' button. The bottom of the screen indicates the form is responsive (1274 x 761) and shows a 'Show hidden' toggle.

From the menu we choose the web resource just created, namely HelloTestFunctions.

The screenshot shows the 'Add JavaScript Library' dialog box in the Power Apps Form editor. The dialog box has a search bar with 'HelloTestFunctions' entered. Below the search bar is a table with the following data:

Display Name	Name	Description
HelloTestFunctions	demo_HelloTestFunctions	

The dialog box also has an 'Add' button and a 'Cancel' button. The background shows the same 'New Expenses' form as in the previous screenshot.

Next, we have On Save and On Load.

The screenshot shows the Power Apps Form Designer interface. The main form is titled 'New Expenses' and contains the following fields: Expense ID, Expense Type, Currency (set to Euro), Expense Amount, Exchange Rate, Status Reason (set to Active), Created By, Modified By, Owner (set to Nicola Paracone), and a Description field. The right sidebar shows the 'Expense Manager Form' with 'On Load' and 'On Save' event handlers.

Choose, for example, On Load and click on + Event Handler. Assuming that the hello_test.js file we have uploaded contains the following two functions

```
function test1(){
    alert("Hello from test1 function")
}

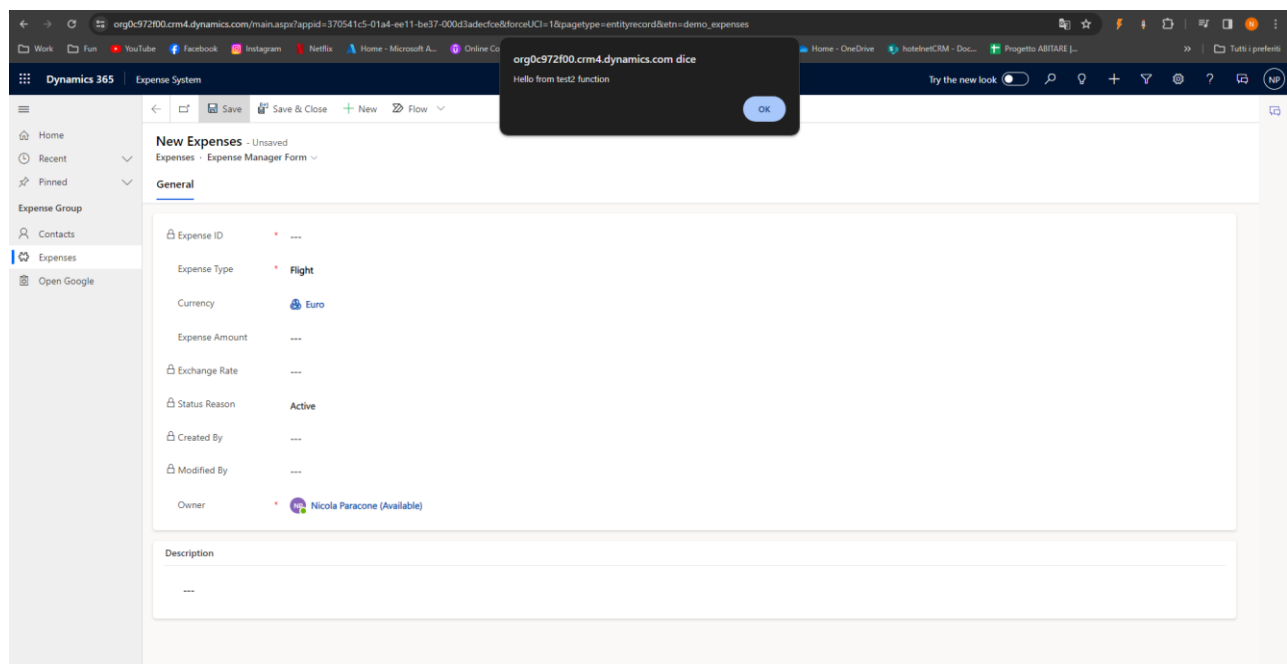
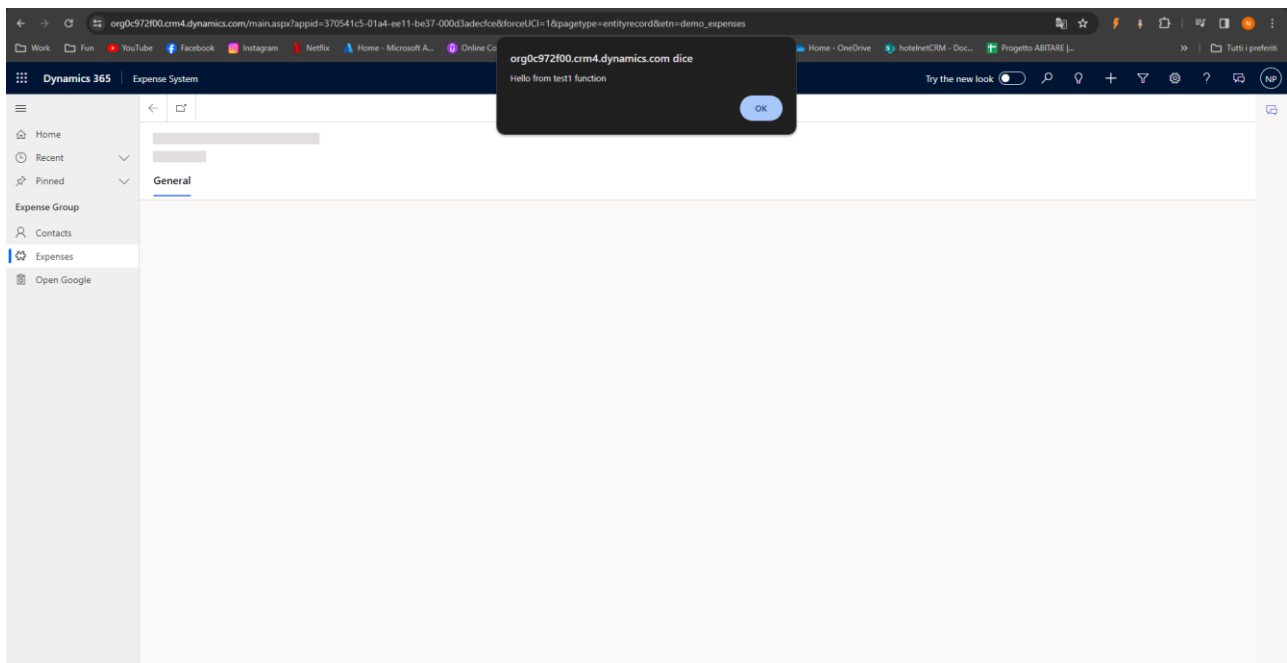
function test2(){
    alert("Hello from test2 function")
}
```

We configure the on-load event to the first function.

The screenshot shows the 'Configure Event' dialog box. The 'Event Type' is set to 'On Load'. The 'Library' is set to 'demo_helloTestFunctions'. The 'Function' is set to 'test1'. The 'Enabled' checkbox is checked. The 'Pass execution context as first parameter' checkbox is unchecked. The 'Comma separated list of parameters that will be passed to the function' field is empty. The 'Done' and 'Cancel' buttons are at the bottom.

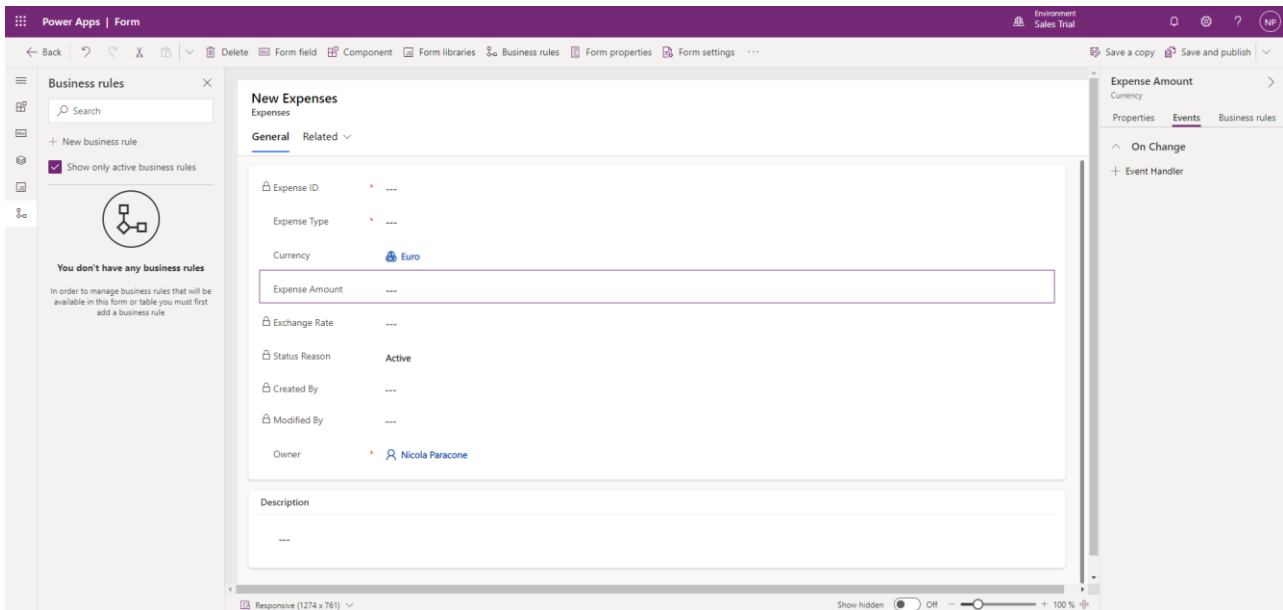
For the on-save event we repeat the same steps, but we choose the second function. Save and publish, then open an expense in Dynamics 365 to test if our web resource is working or not.

The following are the results we should obtain for On Load and On Save, respectively.



Obviously, since we have added this functionality to the Expense Manager Form only, nothing will happen in the Information form.

Similarly, we also call a script on change of the fields. In Power Apps we can choose a field of the form we are editing and under Events we click on + Event Handler.



The logic is the same as before.

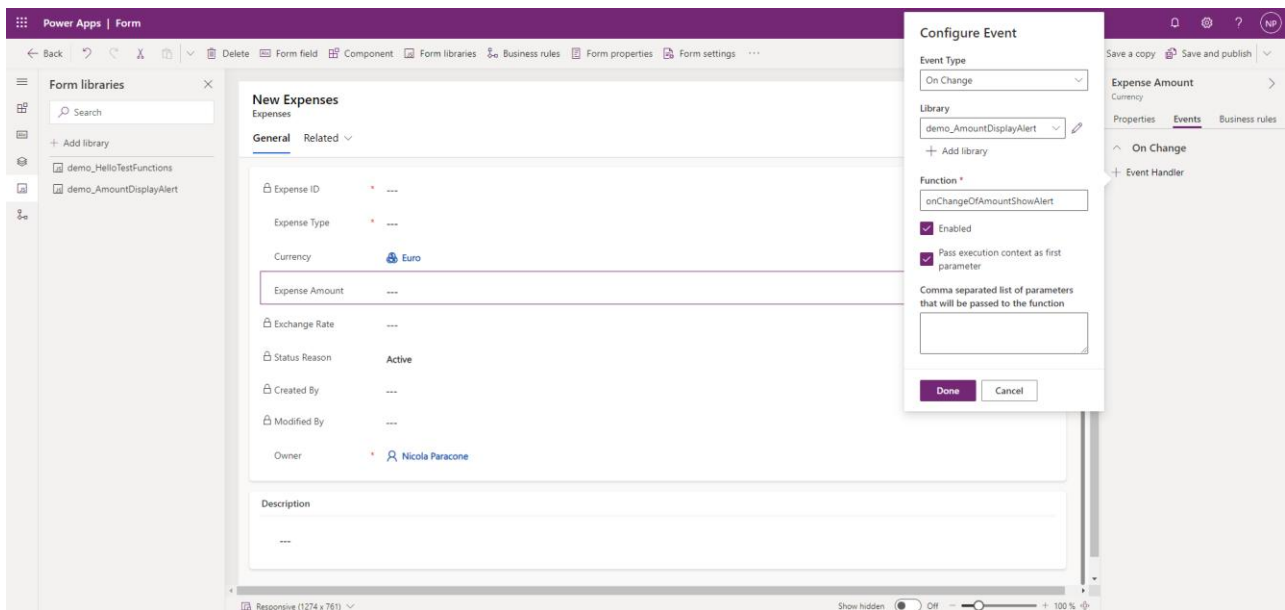
Now we write a script that, when Expense Amount changes, will trigger a pop up displaying the amount of money that the user has inserted. For this we need a way of reading the values in whatever field of the form, which is achieved by using formContext. If we read the documentation at

<https://learn.microsoft.com/en-us/power-apps/developer/model-driven-apps/clientapi/clientapi-form-context>

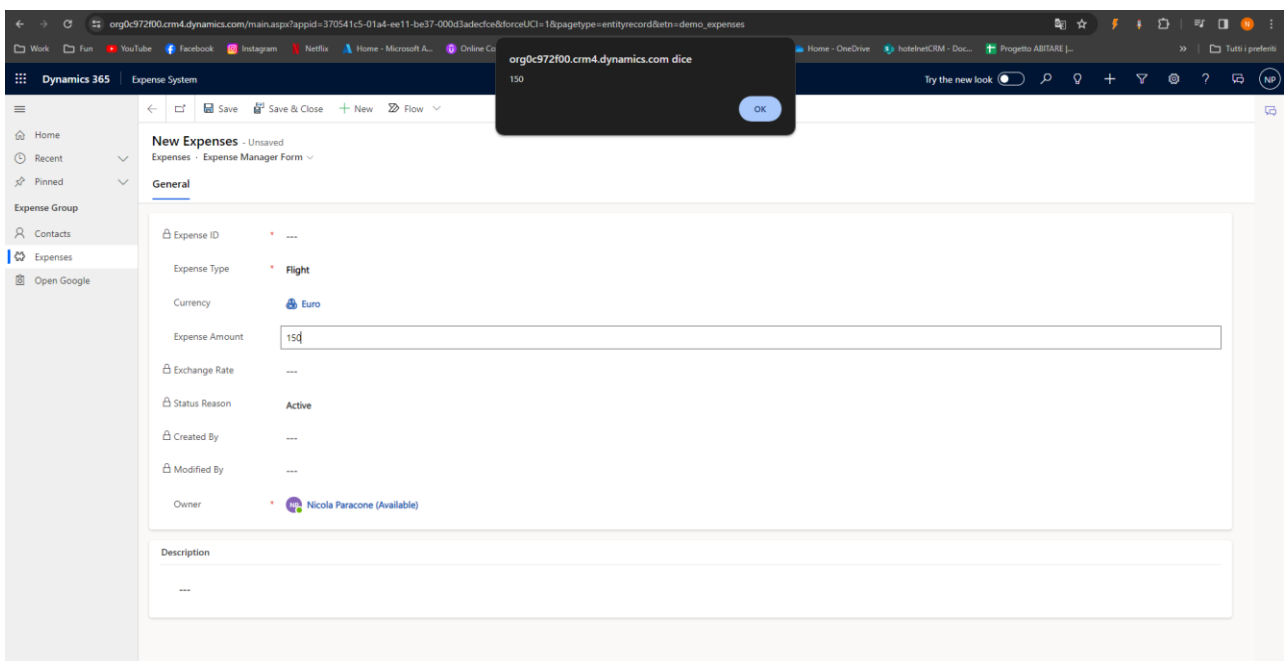
we see that there is another method which uses Xrm.Page.getAttribute. This method, however, will be soon deprecated.

Hence, we focus on formContext. To use it in our function we must pass a parameter called executionContext. In configuring the event we will have to mark the box corresponding to Pass execution context as first parameter and, in passing the names of the fields to the script, we will have to use their logical names (use Power Pane in Dynamics 365 to get them).

```
function onChangeOfAmountShowAlert(executionContext){  
    var formcontext = executionContext.getFormContext();  
    // var expenseamount = Xrm.Page.getAttribute("demo_expenseamount").getValue();  
    var expenseamount = formcontext.getAttribute('demo_expenseamount').getValue();  
    alert(expenseamount)  
}
```



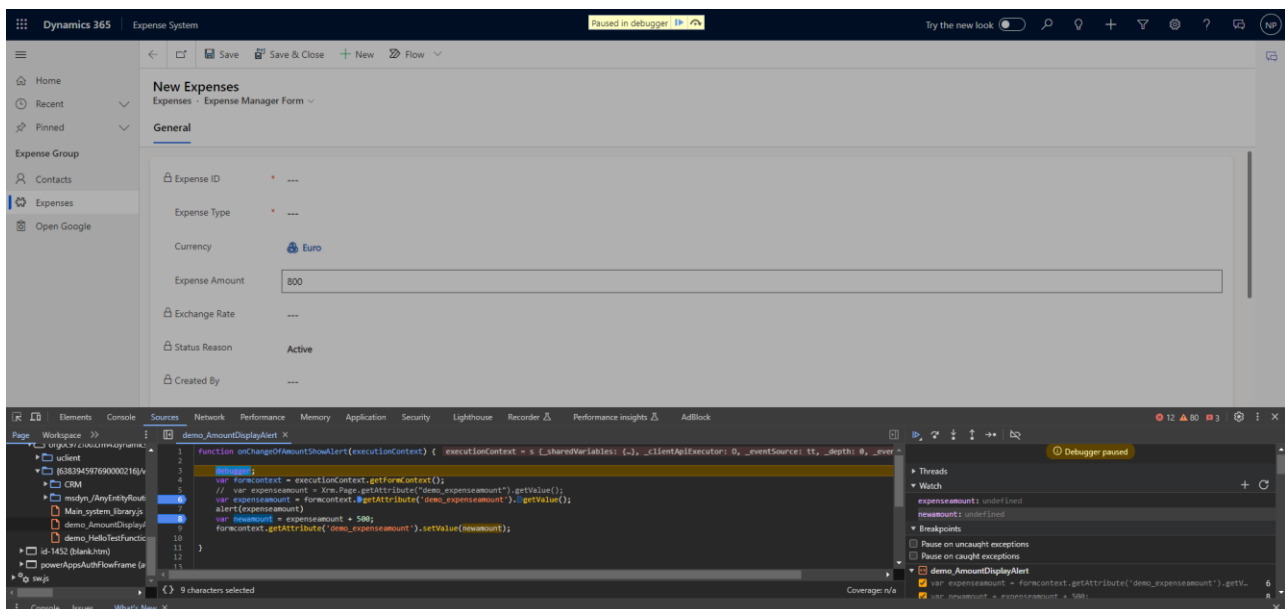
This will be the result in Dynamics 365.



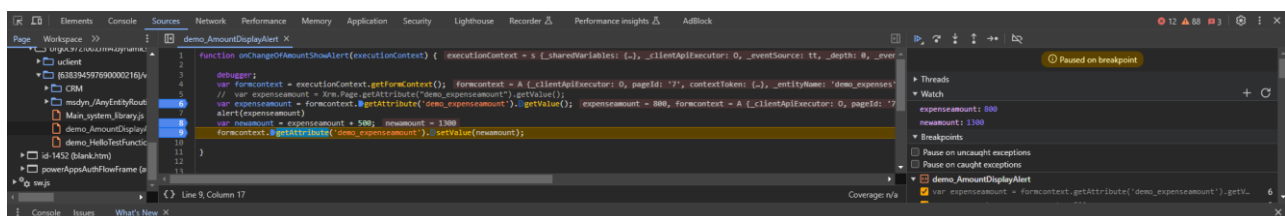
One additional thing we can do, is to insert debugger in our code. In addition, we write a new functionality which adds 500 to the expense amount value inserted by the user.

```
function onChangeOfAmountShowAlert(executionContext){  
    debugger;  
    var formcontext = executionContext.getFormContext();  
    // var expenseamount = Xrm.Page.getAttribute("demo_expenseamount").getValue();  
    var expenseamount = formcontext.getAttribute('demo_expenseamount').getValue();  
    alert(expenseamount)  
    var newamount = expenseamount + 500;  
    formcontext.getAttribute('demo_expenseamount').setValue(newamount);  
}
```

You can access the developer console in Dynamics 365 (or whatever webpage) by pressing Ctrl+Shift+I.



Here it is possible to use tools like those of Visual Studio; for example, we can add breakpoints and variables to Watch, and then check the values they assume during execution.



In the figure below we show how to access the classic mode editor we were talking about before.

